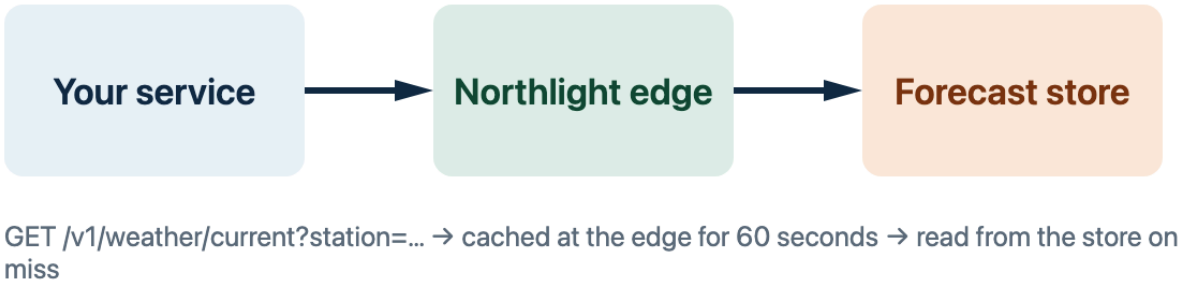
Getting started

Four steps take you from nothing to a first forecast in production: a key, a request, a look at the response, and a polling schedule that respects the cache. The whole flow is plain HTTPS with a key in a header; there is no SDK to install, and any HTTP client will do.



Get a key

Keys are issued per project and scoped to the stations a project may read. A project is created in the Northlight console, and every key it issues carries the project's plan and its station list. Two kinds of key exist:

Kind	Prefix	Purpose
Live	`nlk_live_`	Reads real stations and counts against the project's plan.
Test	`nlk_test_`	Reads the synthetic station `TEST-01` only, returns fixed data and is never rate limited.

Start with a test key. `TEST-01` reports the same reading every ten minutes and a forecast that is always 12 °C and calm, so your parser can be checked against known values before a live key is involved.

Requirements

Step	Prerequisites	
Register	• A project name • A billing contact • At least one station	• Accepted terms • A callback URL

Step	Prerequisites
Rotate	• The current key • Owner role
Revoke	• Owner role

Store the key in your secret manager and expose it to your service as an environment variable; the examples below read `NORTHLIGHT_KEY`.

> **Keep the key out of the browser**
>
> A key grants every scope its project holds. Call Northlight from your own backend and pass results on; never ship the key in client-side code, a mobile app or a public repository. If a key leaks, revoke it in the console and issue a new one; requests with a revoked key fail with `key_revoked`.

Make a request New

Send the key in the `Authorization` header and ask for a station by its code:

```
curl -H "Authorization: Bearer $NORTHLIGHT_KEY" \
  "https://api.northlight.example/v1/weather/current?station=OSL-01"
```

```
{
  "station": "OSL-01",
  "observedAt": "2026-09-22T06:00:00Z",
  "temperature": 11.4,
  "windSpeed": 3.2,
  "windDirection": 240,
  "humidity": 87,
  "pressure": 1009.6,
  "precipitation": 0.0,
  "conditions": "overcast",
  "stale": false,
  "units": {
    "temperature": "C",
    "windSpeed": "m/s",
    "pressure": "hPa",
    "precipitation": "mm"
```

```
  }
}
```

The same request from a Node.js service, with the error envelope handled:

```
const response = await fetch(
  "https://api.northlight.example/v1/weather/current?station=OSL-01",
  { headers: { Authorization: `Bearer ${process.env.NORTHLIGHT_KEY}` } },
)
const body = await response.json()
if (!response.ok) throw new Error(`${body.error.code}: ${body.error.message}`)
console.log(`${body.station}: ${body.temperature} °${body.units.temperature}`)
```

And from Python:

```
import os
import requests

response = requests.get(
    "https://api.northlight.example/v1/weather/current",
    params={"station": "OSL-01"},
    headers={"Authorization": f"Bearer {os.environ['NORTHLIGHT_KEY']}"},
    timeout=5,
)
body = response.json()
if not response.ok:
    raise RuntimeError(f"{body['error']['code']}: {body['error']['message']}")
print(body["station"], body["temperature"], body["units"]["temperature"])
```

Read the response

Every response carries `observedAt`, the instant the station reported[1], and a `units` object naming the unit of every numeric field. The fields you will use most:

Field	Type	Description
`observedAt`	string	When the station made the reading, ISO 8601 in UTC.
`temperature`	number	Air temperature two metres above ground.
`windSpeed`	number	Ten-minute mean wind speed; gusts are in `windGust` when the station has it.

Field	Type	Description
`windDirection`	integer	Degrees clockwise from north, the direction the wind comes from.
`conditions`	string	One of `clear`, `partly-cloudy`, `overcast`, `fog`, `rain`, `snow`, `sleet`.
`stale`	boolean	`true` when the reading is older than fifteen minutes.

A missing station returns `404` with a machine-readable `code`, listed under Error codes. Branch on the code; the `message` is for humans and may change between releases.

Two response headers matter from the first request on. `X-Limit-Minute` and `X-Limit-Hour` report how many requests remain in the current windows, and `ETag` identifies the reading, so a later request can send `If-None-Match` and receive a `304` that does not count against any window:

```
GET /v1/weather/current?station=OSL-01 HTTP/1.1
Host: api.northlight.example
Authorization: Bearer nlk_live_…
If-None-Match: "obs-OSL-01-1758520800"
```

Stay within the limits

Current conditions are cached at the edge for sixty seconds, so polling faster than that returns the same reading and spends your quota. The windows and the headers that report them are in Rate limits; the short version is below.

Windows

As a new integrator you share one rule: a key has a rolling minute window on `/v1/weather/current` and a rolling hour window across all endpoints, the size of each depends on the plan, and both counters are reported on every response. A request that finds either window empty is answered with `429 rate_limited` and is not counted.

Header	Meaning
`X-Limit-Minute`	Requests left in the current minute window.
`X-Limit-Hour`	Requests left in the current hour window.
`Retry-After`	Seconds to wait, present only on a `429`.

Windows are rolling, not aligned to the clock: a request made at 12:00:30 leaves the minute window at 12:01:30. A `304` answer to a conditional request counts against neither window, and neither does a `429` or a `5xx`.

A schedule that works for most services:

1. Poll `/v1/weather/current` once a minute per station, offset by a few seconds per station so the requests do not all land at once.
2. Fetch the hourly forecast once an hour, shortly after the model run lands at five past the hour.
3. Send `If-None-Match` on every poll and treat a `304` as "nothing new".
4. On a `429`, wait for `Retry-After`; on a `503`, back off as described in Backoff.

> **Bursts are counted per key**
>
> Two services sharing one key share one window. Give each service its own key so a spike in one cannot starve the other, and so a leaked key can be revoked without stopping everything else.

Go to production

Before switching from the test key to a live one, check that your client:

- reads `units` instead of assuming metric;
- treats `stale: true` as a reason to show the reading's age, not to drop it;
- handles every code in Error codes, including `station_retired`, which arrives one day for any long-running client;
- logs `X-Request-Id` from every response, so support can find the request;
- ignores fields it does not know, since additive changes ship without notice (see Versioning).

> **Ready for a live key**
>
> A client that passes the list above moves to a live key by changing one environment variable. Nothing else in the request changes between the two kinds of key.

Footnotes

1. Always UTC, always with a `Z` suffix. Convert at the edge of your own system, not in the data layer; the station's IANA time zone is in its detail record. ↩